

Funções e Bibliotecas em Python

Sebenta melhorada e expandida

Tema: funções, parâmetros, return, validação, tratamento de erros, objetos mutáveis, recursão, funções lambda, programação funcional, bibliotecas, módulos, pacotes e modularização em Python.

Objetivo da sebenta: ajudar o estudante a escrever programas mais organizados, reutilizáveis e fáceis de manter, usando funções próprias e bibliotecas já existentes.

Índice

1. Introdução às funções
 2. Criar e chamar funções
 3. `print()` e `return`
 4. Parâmetros e argumentos
 5. Âmbito das variáveis: variáveis locais e globais
 6. Validação e tratamento de erros
 7. Objetos mutáveis e imutáveis
 8. Boas práticas na criação de funções
 9. Docstrings, type hints e documentação
 10. Funções recursivas
 11. Funções lambda
 12. Programação funcional
 13. Bibliotecas, módulos e pacotes
 14. Bibliotecas da biblioteca padrão
 15. Bibliotecas externas
 16. Criar módulos próprios
 17. Modularização de programas
 18. Testar funções
 19. Estudo de caso: gestão de notas
 20. Resumo geral
 21. Exercícios propostos
 22. Soluções orientadoras
 23. Anexos
-

1. Introdução às funções

1.1 O que é uma função?

Uma **função** é um bloco de código reutilizável que executa uma tarefa específica.

Em Python, as funções permitem dividir um programa em partes menores. Cada parte fica responsável por uma tarefa clara.

As funções tornam o código:

- mais organizado;
- mais reutilizável;
- mais fácil de compreender;
- mais fácil de corrigir;
- mais simples de testar;
- menos repetitivo.

Exemplo simples:

```
def mostrar_mensagem():  
    print("Bem-vindo à aula de Python!")
```

```
mostrar_mensagem()
```

Neste exemplo:

- def cria a função;
 - mostrar_mensagem é o nome da função;
 - os parênteses () indicam que se trata de uma função;
 - o código indentado pertence ao corpo da função;
 - a função só executa quando é chamada.
-

1.2 Porque usamos funções?

Sem funções, o programador tende a repetir código muitas vezes.

Exemplo sem função:

```
print(5 * 5)  
print(8 * 8)  
print(10 * 10)
```

Exemplo com função:

```
def quadrado(x):  
    return x * x
```

```
print(quadrado(5))  
print(quadrado(8))  
print(quadrado(10))
```

A segunda abordagem é melhor porque a lógica do cálculo do quadrado fica definida uma só vez.

1.3 Estrutura geral de uma função

```
def nome_da_funcao(parametros):  
    instrucoes  
    return resultado
```

Componentes principais:

Elemento	Função
def	Palavra-chave usada para definir funções.
Nome da função	Identifica a tarefa executada.
Parâmetros	Valores que a função pode receber.
Corpo da função	Bloco de código indentado.
return	Devolve um resultado ao programa.

Exemplo:

```
def somar(a, b):  
    resultado = a + b  
    return resultado
```

1.4 Funções como forma de pensar

Uma boa função responde a uma pergunta simples:

“Que tarefa específica esta parte do programa deve executar?”

Exemplos de boas tarefas para funções:

- calcular uma média;
 - validar uma idade;
 - converter graus Celsius para Fahrenheit;
 - contar palavras num texto;
 - verificar se uma palavra é palíndromo;
 - mostrar um menu;
 - ler dados do utilizador;
 - guardar dados num ficheiro.
-

2. Criar e chamar funções

2.1 Definir uma função

Definir uma função significa dar um nome a um conjunto de instruções.

```
def saudacao():  
    print("Olá!")
```

A função foi criada, mas ainda não executou.

2.2 Chamar uma função

Para executar uma função, usa-se o nome seguido de parênteses:

```
saudacao()
```

Exemplo completo:

```
def mostrar():  
    print("Dentro da função")  
  
print("Antes")  
mostrar()  
print("Depois")
```

Saída:

```
Antes  
Dentro da função  
Depois
```

2.3 A ordem importa

Em Python, a função tem de estar definida antes de ser chamada.

Erro:

```
mostrar_mensagem()
```

```
def mostrar_mensagem():  
    print("Olá")
```

Correto:

```
def mostrar_mensagem():  
    print("Olá")
```

```
mostrar_mensagem()
```

Nota: em programas maiores, é comum definir as funções primeiro e chamar a função principal no fim.

2.4 Erros comuns ao criar funções

Falta de indentação

```
def teste():  
print("Erro")
```

Correto:

```
def teste():  
    print("Certo")
```

Falta de dois pontos

```
def teste()  
    print("Erro")
```

Correto:

```
def teste():  
    print("Certo")
```

Esquecer os parênteses na chamada

```
teste
```

Isto não executa a função. Apenas faz referência ao objeto função.

Correto:

```
teste()
```

3. print() e return

3.1 Procedimentos e funções

Em muitos contextos escolares, costuma fazer-se a seguinte distinção:

- **procedimento:** executa uma ação, normalmente usando `print()`;
- **função:** devolve um valor, normalmente usando `return`.

Em Python, tecnicamente tudo é função, mas esta distinção ajuda a estudar.

3.2 Função que apenas mostra informação

```
def mostrar_numero():  
    print(10)
```

```
mostrar_numero()
```

Esta função mostra o número 10, mas não devolve esse valor ao programa.

3.3 Função que devolve informação

```
def dobro(x):  
    return x * 2
```

```
resultado = dobro(5)  
print(resultado)
```

Aqui, o valor devolvido por return pode ser guardado na variável resultado.

3.4 Diferença entre print() e return

print()	return
Mostra informação no ecrã. Serve para apresentação. Não permite guardar diretamente o resultado. Pode ser usado várias vezes numa função.	Devolve informação ao programa. Serve para reutilização. Permite guardar e reutilizar o resultado. Normalmente termina a execução da função.

Exemplo:

```
def exemplo_print():  
    print(5)
```

```
def exemplo_return():  
    return 5
```

```
valor = exemplo_return()  
print(valor)
```

3.5 Quando usar print()?

Usa print() quando queres apresentar uma mensagem ao utilizador.

```
def mostrar_boas_vindas():  
    print("Bem-vindo ao programa!")
```

3.6 Quando usar return?

Usa return quando queres que o resultado possa ser usado noutra cálculo.

```
def calcular_media(a, b):  
    return (a + b) / 2
```

```
media = calcular_media(10, 14)  
final = media + 1  
print(final)
```

3.7 O que acontece sem return?

Quando uma função não tem return, Python devolve automaticamente None.

```
def saudacao():  
    print("Olá")
```

```
resultado = saudacao()  
print(resultado)
```

Saída:

```
Olá  
None
```

4. Parâmetros e argumentos

4.1 Conceitos

Parâmetros são as variáveis escritas na definição da função.

Argumentos são os valores enviados quando a função é chamada.

```
def media(a, b):  
    return (a + b) / 2
```

```
print(media(10, 14))
```

Neste exemplo:

- a e b são parâmetros;
 - 10 e 14 são argumentos.
-

4.2 Argumentos posicionais

São argumentos passados pela ordem.

```
def apresentar(nome, idade):  
    print(f"{nome} tem {idade} anos.")
```

```
apresentar("Ana", 16)
```

A ordem é importante.

```
apresentar(16, "Ana") # resultado estranho
```

4.3 Argumentos nomeados

São argumentos passados pelo nome do parâmetro.

```
def apresentar(nome, idade):  
    print(f"{nome} tem {idade} anos.")
```

```
apresentar(nome="Ana", idade=16)  
apresentar(idade=16, nome="Ana")
```

Com argumentos nomeados, a ordem deixa de ser tão importante.

4.4 Parâmetros com valores por defeito

Uma função pode ter valores predefinidos.

```
def saudacao(nome="aluno"):  
    print(f"Olá, {nome}!")
```

```
saudacao()  
saudacao("Rita")
```

Saída:

```
Olá, aluno!  
Olá, Rita!
```

4.5 Mistura de argumentos posicionais e nomeados

```
def criar_utilizador(nome, idade, ativo=True):  
    return {  
        "nome": nome,  
        "idade": idade,  
        "ativo": ativo  
    }
```

```
u1 = criar_utilizador("João", 17)  
u2 = criar_utilizador("Maria", 18, ativo=False)
```

Regra importante: argumentos posicionais devem aparecer antes dos argumentos nomeados.

4.6 Número variável de argumentos: *args

*args permite receber vários argumentos posicionais.

```
def somar_todos(*numeros):  
    total = 0  
    for numero in numeros:  
        total += numero  
    return total
```

```
print(somar_todos(1, 2, 3))
print(somar_todos(10, 20, 30, 40))
```

numeros será uma tupla.

4.7 Número variável de argumentos nomeados: ****kwargs**

****kwargs** permite receber vários argumentos nomeados.

```
def mostrar_dados(**dados):
    for chave, valor in dados.items():
        print(chave, "=", valor)
```

```
mostrar_dados(nome="Ana", idade=16, turma="10A")
```

`dados` será um dicionário.

4.8 Desempacotar listas, tuplos e dicionários

```
def somar(a, b, c):
    return a + b + c
```

```
valores = [1, 2, 3]
print(somar(*valores))
```

Com dicionários:

```
def apresentar(nome, idade):
    print(f"{nome} tem {idade} anos.")
```

```
dados = {"nome": "Ana", "idade": 16}
apresentar(**dados)
```

4.9 Exemplos práticos com parâmetros

```
def dobro(x):
    return x * 2
```

```
def potencia(base, expoente):
    return base ** expoente
```

```
def media(a, b):
    return (a + b) / 2
```

```
def fatorial(n):
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado
```

4.10 Erros frequentes com argumentos

Número incorreto de argumentos

```
def media(a, b):  
    return (a + b) / 2
```

```
media(10) # erro
```

Tipo inválido

```
def dobro(x):  
    return x * 2
```

```
print(dobro("abc")) # não dá erro, mas pode não ser o comportamento desejado
```

O resultado será:

abcbcb

Por isso, em alguns programas, pode ser necessário validar tipos.

5. Âmbito das variáveis: variáveis locais e globais

5.1 Variáveis locais

Uma variável criada dentro de uma função é, normalmente, local à função.

```
def calcular():  
    resultado = 10  
    return resultado
```

```
print(calcular())
```

A variável resultado só existe dentro da função.

5.2 Variáveis globais

Uma variável criada fora das funções é global.

```
mensagem = "Olá"
```

```
def mostrar():  
    print(mensagem)
```

```
mostrar()
```

Funções conseguem ler variáveis globais, mas alterar variáveis globais diretamente pode tornar o programa confuso.

5.3 Evitar alterar variáveis globais

Exemplo pouco recomendado:

```
contador = 0
```

```
def aumentar():  
    global contador  
    contador += 1
```

Alternativa melhor:

```
def aumentar(contador):  
    return contador + 1
```

```
contador = 0  
contador = aumentar(contador)
```

Esta versão é mais previsível, porque a função recebe um valor e devolve um novo valor.

5.4 Regra LEGB

Quando Python procura uma variável, segue normalmente esta ordem:

Letra	Significado	Descrição
L	Local	Dentro da função atual.
E	Enclosing	Função exterior, no caso de funções dentro de funções.
G	Global	Ficheiro atual.
B	Built-in	Nomes próprios do Python, como print, len, sum.

Exemplo:

```
x = "global"
```

```
def exterior():  
    x = "exterior"  
  
    def interior():  
        x = "local"  
        print(x)  
  
    interior()
```

```
exterior()
```

Saída:

```
local
```

6. Validação e tratamento de erros

6.1 Porque validar dados?

Programas reais recebem dados imperfeitos. Um utilizador pode introduzir texto onde era esperado um número, uma lista pode estar vazia ou uma idade pode ser negativa.

A validação melhora:

- robustez;
 - segurança;
 - previsibilidade;
 - qualidade do programa.
-

6.2 Validação simples com if

```
def validar_idade(idade):  
    if idade < 0:  
        return "Idade inválida"  
    return "Idade válida"  
  
print(validar_idade(20))  
print(validar_idade(-5))
```

Esta abordagem é simples, mas mistura validação com mensagens.

6.3 Validação com True ou False

```
def idade_valida(idade):  
    return idade >= 0  
  
if idade_valida(18):  
    print("Idade aceite")  
else:  
    print("Idade inválida")
```

Esta abordagem permite reutilizar melhor a função.

6.4 Validação com exceções

Uma exceção é um erro que pode ser tratado pelo programa.

```
def validar_idade(idade):  
    if idade < 0:  
        raise ValueError("A idade não pode ser negativa")
```

Se a idade for inválida, a função lança um erro.

6.5 Exceções comuns

Exceção	Quando acontece
TypeError	Tipo de dado errado.
ValueError	Valor inválido.
ZeroDivisionError	Divisão por zero.
IndexError	Índice inexistente numa lista.
KeyError	Chave inexistente num dicionário.
FileNotFoundError	Ficheiro não encontrado.

6.6 try e except

```
try:
    idade = int(input("Idade: "))
    if idade < 0:
        raise ValueError("Idade inválida")
    print("Idade aceite")
except ValueError as erro:
    print("Erro:", erro)
```

O bloco try tenta executar código.

O bloco except trata o erro.

6.7 else e finally

```
try:
    numero = int(input("Número: "))
except ValueError:
    print("Valor inválido")
else:
    print("Número lido:", numero)
finally:
    print("Fim da tentativa")
```

Bloco	Quando executa
try	Sempre que se tenta executar o código.
except	Quando ocorre erro.
else	Quando não ocorre erro.
finally	Sempre, com ou sem erro.

6.8 Exemplo: divisão segura

```
def dividir(a, b):
    if b == 0:
        raise ZeroDivisionError("Não é possível dividir por zero")
    return a / b

try:
    resultado = dividir(10, 0)
except ZeroDivisionError as erro:
    print("Erro:", erro)
```

6.9 Comparação entre validação simples e exceções

Validação simples	Exceções
Mais simples para iniciantes. Devolve mensagens ou booleanos. Útil quando o erro é esperado e frequente.	Mais adequada para programas maiores. Lança erros específicos. Útil quando se quer separar erro e lógica principal.

7. Objetos mutáveis e imutáveis

7.1 Objetos imutáveis

Objetos imutáveis não podem ser alterados diretamente depois de criados.

Exemplos:

- int;
- float;
- str;
- bool;
- tuple.

```
numero = 10
outro = numero
numero += 1
```

```
print(numero) # 11
print(outro)  # 10
```

7.2 Objetos mutáveis

Objetos mutáveis podem ser alterados depois de criados.

Exemplos:

- list;
- dict;
- set.

```
lista = [1, 2]
outra = lista
lista.append(3)
```

```
print(lista)
print(outra)
```

Saída:

```
[1, 2, 3]
[1, 2, 3]
```

As duas variáveis apontam para a mesma lista.

7.3 Mutabilidade em funções

```
def adicionar(lista):  
    lista.append(10)
```

```
valores = [1, 2]  
adicionar(valores)  
print(valores)
```

A lista original foi alterada.

7.4 Evitar efeitos secundários inesperados

Um efeito secundário acontece quando uma função altera algo fora dela.

Exemplo com efeito secundário:

```
def adicionar_dez(lista):  
    lista.append(10)  
    return lista
```

Versão mais segura:

```
def adicionar_dez(lista):  
    nova_lista = lista.copy()  
    nova_lista.append(10)  
    return nova_lista
```

7.5 Erro clássico: listas como valores por defeito

Evita isto:

```
def adicionar_item(item, lista=[]):  
    lista.append(item)  
    return lista
```

O mesmo objeto lista pode ser reutilizado entre chamadas.

Melhor:

```
def adicionar_item(item, lista=None):  
    if lista is None:  
        lista = []  
    lista.append(item)  
    return lista
```

8. Boas práticas na criação de funções

8.1 Nomes claros

O nome da função deve indicar a ação.

Pouco claro:

```
def f(x):  
    return x * 2
```

Melhor:

```
def calcular_dobro(numero):  
    return numero * 2
```

8.2 Uma função deve ter uma responsabilidade principal

Pouco recomendado:

```
def processar(notas):  
    media = sum(notas) / len(notas)  
    print("Média:", media)  
    if media >= 10:  
        print("Aprovado")  
    else:  
        print("Reprovado")
```

Melhor:

```
def calcular_media(notas):  
    return sum(notas) / len(notas)  
  
def classificar(media):  
    if media >= 10:  
        return "Aprovado"  
    return "Reprovado"  
  
def mostrar_resultado(media, classificacao):  
    print("Média:", media)  
    print("Classificação:", classificacao)
```

8.3 Separar cálculo e apresentação

Cálculo:

```
def calcular_area_retangulo(largura, altura):  
    return largura * altura
```

Apresentação:

```
def mostrar_area(area):  
    print(f"Área: {area}")
```

Esta separação facilita testes e reutilização.

8.4 Evitar repetição

Se o mesmo código aparece várias vezes, provavelmente deve virar função.

Antes:

```
media1 = (12 + 14 + 15) / 3  
media2 = (10 + 11 + 13) / 3
```

Depois:

```
def calcular_media(notas):  
    return sum(notas) / len(notas)  
  
media1 = calcular_media([12, 14, 15])  
media2 = calcular_media([10, 11, 13])
```

8.5 Não criar funções demasiado grandes

Uma função muito grande é difícil de testar e compreender.

Sinais de que uma função deve ser dividida:

- tem muitas linhas;
 - faz muitas tarefas diferentes;
 - tem muitos if e for misturados;
 - o nome da função é vago, como `processar_tudo`;
 - é difícil explicar a função numa frase.
-

8.6 Checklist de uma boa função

Antes de considerar uma função terminada, confirma:

- o nome é claro?
 - os parâmetros são necessários?
 - a função devolve algo útil?
 - há validação dos dados importantes?
 - a função faz apenas uma tarefa principal?
 - há testes simples para verificar o resultado?
 - o código é legível?
-

9. Docstrings, type hints e documentação

9.1 Docstrings

Uma docstring é uma pequena descrição colocada no início da função.

```
def calcular_media(notas):  
    """Calcula a média aritmética de uma lista de notas."""  
    return sum(notas) / len(notas)
```

As docstrings ajudam a explicar o objetivo da função.

9.2 Docstring mais completa

```
def calcular_media(notas):  
    """  
    Calcula a média aritmética de uma lista de notas.  
  
    Parâmetros:  
    notas: lista de números.
```


Devolve:
A média das notas.

Lança:
ValueError: se a lista estiver vazia.
"""
if len(notas) == 0:
 raise ValueError("A lista de notas não pode estar vazia")
return sum(notas) / len(notas)

9.3 Type hints

Type hints indicam os tipos esperados.

```
def somar(a: int, b: int) -> int:  
    return a + b
```

Isto não obriga automaticamente os tipos em tempo de execução, mas torna o código mais claro.

9.4 Type hints com listas

```
def calcular_media(notas: list[float]) -> float:  
    if len(notas) == 0:  
        raise ValueError("A lista não pode estar vazia")  
    return sum(notas) / len(notas)
```

9.5 Exemplo completo com docstring e type hints

```
def converter_celsius_para_fahrenheit(celsius: float) -> float:  
    """Converte uma temperatura de Celsius para Fahrenheit."""  
    return celsius * 9 / 5 + 32  
  
print(converter_celsius_para_fahrenheit(20))
```

10. Funções recursivas

10.1 Conceito

Uma função recursiva é uma função que se chama a si própria.

A recursão precisa sempre de:

- um **caso base**, que termina a repetição;
 - um **caso recursivo**, que chama a função novamente.
-

10.2 Fatorial com recursão

```
def fatorial(n):  
    if n == 0:  
        return 1  
    return n * fatorial(n - 1)
```

```
print(fatorial(4))
```

Explicação:

```
fatorial(4)  
= 4 * fatorial(3)  
= 4 * 3 * fatorial(2)  
= 4 * 3 * 2 * fatorial(1)  
= 4 * 3 * 2 * 1 * fatorial(0)  
= 24
```

10.3 Fatorial com validação

```
def fatorial(n):  
    if n < 0:  
        raise ValueError("0 fatorial não existe para números negativos")  
    if n == 0:  
        return 1  
    return n * fatorial(n - 1)
```

10.4 Soma de uma lista com recursão

```
def somar_lista(lista):  
    if len(lista) == 0:  
        return 0  
    return lista[0] + somar_lista(lista[1:])  
  
print(somar_lista([1, 2, 3, 4]))
```

10.5 Recursão vs ciclo

Muitos problemas podem ser resolvidos com recursão ou com ciclos.

Fatorial com ciclo:

```
def fatorial_iterativo(n):  
    resultado = 1  
    for i in range(1, n + 1):  
        resultado *= i  
    return resultado
```

Fatorial com recursão:

```
def fatorial_recursivo(n):  
    if n == 0:  
        return 1  
    return n * fatorial_recursivo(n - 1)
```

A versão iterativa costuma ser mais simples para iniciantes e evita problemas com recursão profunda.

10.6 Limitações da recursão

A recursão pode:

- consumir mais memória;
 - ser mais difícil de entender;
 - gerar RecursionError se não houver caso base correto;
 - ser menos eficiente em alguns problemas.
-

11. Funções lambda

11.1 Conceito

Uma função lambda é uma função pequena e anônima.

Sintaxe:

lambda argumentos: expressão

Exemplo:

```
dobro = lambda x: x * 2
print(dobro(5))
```

11.2 Quando usar lambda?

Use lambda para funções muito curtas, normalmente passadas como argumento a outras funções.

Exemplo:

```
numeros = [1, 2, 3, 4]
quadrados = list(map(lambda x: x ** 2, numeros))
print(quadrados)
```

11.3 Diferença entre def e lambda

def	lambda
Cria funções completas.	Cria funções curtas e anônimas.
Pode ter várias instruções.	Tem apenas uma expressão.
Melhor para lógica complexa.	Melhor para uso rápido.
Pode ter docstring.	Não é ideal para documentação.

11.4 Exemplo com ordenação

```
alunos = [
    {"nome": "Ana", "nota": 15},
    {"nome": "Bruno", "nota": 12},
    {"nome": "Carla", "nota": 18},
]

ordenados = sorted(alunos, key=lambda aluno: aluno["nota"])
print(ordenados)
```

12. Programação funcional

12.1 Conceito

A programação funcional é uma forma de programar que dá destaque a funções, transformação de dados e redução de efeitos secundários.

Em Python, podemos usar algumas ferramentas funcionais, como:

- `map()`;
 - `filter()`;
 - `reduce()`;
 - `sorted()` com `key`;
 - list comprehensions;
 - funções puras.
-

12.2 Funções puras

Uma função pura:

- devolve sempre o mesmo resultado para os mesmos argumentos;
- não altera variáveis externas;
- não modifica listas ou dicionários recebidos, a não ser que isso seja claramente esperado.

Exemplo de função pura:

```
def calcular_dobro(numero):
    return numero * 2
```

Exemplo com efeito secundário:

```
def adicionar(lista, valor):
    lista.append(valor)
```

12.3 `map()`

`map()` aplica uma função a cada elemento de uma sequência.

```
numeros = [1, 2, 3]
resultado = list(map(lambda x: x * 2, numeros))
print(resultado)
```

Saída:

```
[2, 4, 6]
```

Versão com list comprehension:

```
numeros = [1, 2, 3]
resultado = [x * 2 for x in numeros]
print(resultado)
```

12.4 filter()

filter() seleciona elementos que cumprem uma condição.

```
numeros = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares)
```

Versão com list comprehension:

```
pares = [x for x in numeros if x % 2 == 0]
```

12.5 reduce()

reduce() combina os elementos de uma sequência até obter um único resultado.

```
from functools import reduce
```

```
numeros = [1, 2, 3, 4]
soma = reduce(lambda x, y: x + y, numeros)
print(soma)
```

Neste caso, também se poderia usar simplesmente:

```
soma = sum(numeros)
```

12.6 sorted() com função key

```
palavras = ["casa", "programacao", "sol", "python"]
ordenadas = sorted(palavras, key=len)
print(ordenadas)
```

Ordenação por nota:

```
alunos = [
    ("Ana", 15),
    ("Bruno", 12),
    ("Carla", 18),
]
```

```
ordenados = sorted(alunos, key=lambda aluno: aluno[1])
print(ordenados)
```

12.7 Procedimental vs funcional

Procedimental	Funcional
Foco em passos. Usa muitos ciclos explícitos. Pode alterar variáveis ao longo do programa. Mais intuitivo para iniciantes.	Foco em transformação de dados. Usa funções e expressões. Tende a evitar efeitos secundários. Pode tornar o código mais compacto.

13. Bibliotecas, módulos e pacotes

13.1 O que é uma biblioteca?

Uma **biblioteca** é um conjunto de código reutilizável que ajuda a resolver problemas sem escrever tudo do zero.

Exemplo:

```
import math  
  
print(math.sqrt(25))
```

A biblioteca math fornece funções matemáticas.

13.2 O que é um módulo?

Um **módulo** é, normalmente, um ficheiro .py com código Python.

Exemplo:

calculadora.py

Dentro desse ficheiro podem existir funções, variáveis e classes.

13.3 O que é um pacote?

Um **pacote** é uma pasta que agrupa módulos.

Exemplo:

```
meu_pacote/  
  __init__.py  
  matematica.py  
  textos.py
```

O ficheiro `__init__.py` indica que a pasta pode ser tratada como pacote. Em versões modernas de Python, nem sempre é obrigatório, mas continua a ser útil em muitos projetos.

13.4 Formas de importar

Importar o módulo inteiro

```
import math  
  
print(math.sqrt(16))
```

Importar uma função específica

```
from math import sqrt  
  
print(sqrt(16))
```

Importar com alias

```
import statistics as stats  
  
print(stats.mean([10, 12, 14]))
```

13.5 Importar tudo: evitar *

Evita:

```
from math import *
```

Esta forma pode tornar o programa confuso, porque muitos nomes entram no programa sem ser claro de onde vieram.

Melhor:

```
import math
```

ou:

```
from math import sqrt, pi
```

14. Bibliotecas da biblioteca padrão

A biblioteca padrão é o conjunto de módulos que já vem com Python.

14.1 math

Usada para operações matemáticas.

```
import math  
  
print(math.sqrt(25))  
print(math.pi)  
print(math.ceil(4.2))  
print(math.floor(4.8))
```

14.2 random

Usada para gerar valores aleatórios.

```
import random  
  
print(random.randint(1, 10))  
print(random.choice(["Ana", "Bruno", "Carla"]))
```

Exemplo: simular lançamento de dado.

```
import random

def lancar_dado():
    return random.randint(1, 6)

print(lancar_dado())
```

14.3 datetime

Usada para datas e horas.

```
from datetime import datetime

agora = datetime.now()
print(agora)
print(agora.year)
print(agora.month)
print(agora.day)

Exemplo:

from datetime import date

def calcular_idade(ano_nascimento):
    ano_atual = date.today().year
    return ano_atual - ano_nascimento

print(calcular_idade(2008))
```

14.4 statistics

Usada para estatística simples.

```
import statistics

notas = [12, 14, 16, 18]
print(statistics.mean(notas))
print(statistics.median(notas))
```

14.5 pathlib

Usada para trabalhar com caminhos de ficheiros de forma moderna.

```
from pathlib import Path

caminho = Path("dados.txt")
print(caminho.exists())

Ler texto:

from pathlib import Path

caminho = Path("dados.txt")
texto = caminho.read_text(encoding="utf-8")
print(texto)
```


Escrever texto:

```
from pathlib import Path

caminho = Path("saida.txt")
caminho.write_text("Olá, ficheiro!", encoding="utf-8")
```

14.6 json

Usada para trabalhar com dados em formato JSON.

```
import json

dados = {
    "nome": "Ana",
    "idade": 16
}

texto = json.dumps(dados, ensure_ascii=False, indent=4)
print(texto)
```

Converter JSON para dicionário:

```
import json

texto = '{"nome": "Ana", "idade": 16}'
dados = json.loads(texto)
print(dados["nome"])
```

14.7 csv

Usada para ler e escrever ficheiros CSV.

```
import csv

with open("alunos.csv", "w", newline="", encoding="utf-8") as ficheiro:
    escritor = csv.writer(ficheiro)
    escritor.writerow(["nome", "nota"])
    escritor.writerow(["Ana", 15])
    escritor.writerow(["Bruno", 12])
```

14.8 collections

Fornece estruturas úteis, como Counter.

```
from collections import Counter

palavras = ["python", "java", "python", "c"]
contagem = Counter(palavras)
print(contagem)
```

14.9 itertools

Ajuda a criar combinações, permutações e sequências.

```
from itertools import combinations
```

```
valores = [1, 2, 3]  
print(list(combinations(valores, 2)))
```

14.10 functools

Inclui ferramentas para trabalhar com funções.

```
from functools import reduce
```

```
numeros = [1, 2, 3, 4]  
produto = reduce(lambda a, b: a * b, numeros)  
print(produto)
```

15. Bibliotecas externas

15.1 O que são bibliotecas externas?

Bibliotecas externas são bibliotecas que não vêm instaladas por defeito com Python. Normalmente são instaladas com pip.

Exemplos comuns:

- pandas para análise de dados;
 - matplotlib para gráficos;
 - requests para pedidos HTTP;
 - pytest para testes;
 - numpy para cálculo numérico.
-

15.2 Instalar bibliotecas com pip

No terminal:

```
pip install pandas
```

Em alguns sistemas:

```
python -m pip install pandas
```

15.3 Ambientes virtuais

Um ambiente virtual permite isolar as bibliotecas de um projeto.

Criar ambiente virtual:

```
python -m venv .venv
```

Ativar no Windows:

```
.venv\Scripts\activate
```

Ativar em macOS/Linux:

```
source .venv/bin/activate
```

Instalar biblioteca dentro do ambiente:

```
python -m pip install pandas
```

15.4 Ficheiro requirements.txt

Serve para registar as bibliotecas usadas no projeto.

Criar:

```
python -m pip freeze > requirements.txt
```

Instalar bibliotecas a partir do ficheiro:

```
python -m pip install -r requirements.txt
```

15.5 Exemplo com pandas

```
import pandas as pd
```

```
dados = {  
    "nome": ["Ana", "Bruno", "Carla"],  
    "nota": [15, 12, 18]  
}
```

```
tabela = pd.DataFrame(dados)  
print(tabela)  
print(tabela["nota"].mean())
```

15.6 Exemplo com matplotlib

```
import matplotlib.pyplot as plt
```

```
nomes = ["Ana", "Bruno", "Carla"]  
notas = [15, 12, 18]
```

```
plt.bar(nomes, notas)  
plt.title("Notas dos alunos")  
plt.xlabel("Aluno")  
plt.ylabel("Nota")  
plt.show()
```

16. Criar módulos próprios

16.1 Criar um módulo simples

Ficheiro matematica.py:

```
def somar(a, b):  
    return a + b
```

```
def subtrair(a, b):  
    return a - b
```

Ficheiro main.py:

```
import matematica  
  
print(matematica.somar(10, 5))  
print(matematica.subtrair(10, 5))
```

16.2 Importar funções específicas

```
from matematica import somar  
  
print(somar(10, 5))
```

16.3 Usar if __name__ == "__main__"

Este bloco permite que determinado código só execute quando o ficheiro é executado diretamente.

```
def somar(a, b):  
    return a + b  
  
if __name__ == "__main__":  
    print(somar(10, 5))
```

Se este ficheiro for importado noutro programa, o print não será executado automaticamente.

17. Modularização de programas

17.1 O que é modularização?

Modularização é dividir um programa em vários ficheiros ou módulos, cada um com uma responsabilidade.

Vantagens:

- melhora a organização;
 - facilita a manutenção;
 - facilita testes;
 - permite reutilizar código;
 - torna o programa mais profissional.
-

17.2 Estrutura simples recomendada

```
projeto/  
    main.py
```

modelo.py
vista.py
controlador.py

Ficheiro	Responsabilidade
modelo.py	Cálculos e regras de negócio.
vista.py	Mostrar informação e ler dados.
controlador.py	Coordenar a lógica do programa.
main.py	Ponto de entrada do programa.

17.3 Exemplo simples

modelo.py:

```
def calcular_media(notas):  
    return sum(notas) / len(notas)
```

vista.py:

```
def mostrar_media(media):  
    print("Média:", media)
```

controlador.py:

```
from modelo import calcular_media  
from vista import mostrar_media
```

```
def executar():  
    notas = [14, 16, 15]  
    media = calcular_media(notas)  
    mostrar_media(media)
```

main.py:

```
from controlador import executar
```

```
executar()
```

17.4 Evitar dependências circulares

Uma dependência circular acontece quando dois módulos dependem diretamente um do outro.

Exemplo problemático:

```
modelo.py importa controlador.py  
controlador.py importa modelo.py
```

Isto pode gerar erros difíceis de compreender.

Regra prática:

- modelo.py não deve depender da vista;
 - vista.py não deve fazer cálculos principais;
 - controlador.py coordena os outros módulos.
-

18. Testar funções

18.1 Porque testar?

Testar uma função é confirmar se ela produz o resultado esperado.

Exemplo:

```
def dobro(x):  
    return x * 2  
  
assert dobro(5) == 10  
assert dobro(0) == 0  
assert dobro(-3) == -6
```

Se os testes passarem, nada aparece. Se falharem, Python mostra um erro.

18.2 Testes simples com assert

```
def calcular_media(notas):  
    if len(notas) == 0:  
        raise ValueError("A lista não pode estar vazia")  
    return sum(notas) / len(notas)  
  
assert calcular_media([10, 20]) == 15  
assert calcular_media([12, 14, 16]) == 14
```

18.3 Testar exceções de forma simples

```
def dividir(a, b):  
    if b == 0:  
        raise ZeroDivisionError("Divisão por zero")  
    return a / b  
  
try:  
    dividir(10, 0)  
except ZeroDivisionError:  
    print("Teste passou: exceção lançada")  
else:  
    print("Teste falhou: exceção não foi lançada")
```

18.4 Organização de testes

Num projeto maior, pode existir uma pasta de testes:

```
projeto/  
    main.py  
    modelo.py  
    tests/  
        test_modelo.py
```

Mesmo sem usar ferramentas avançadas, criar pequenos testes ajuda a evitar erros.

19. Estudo de caso: gestão de notas

19.1 Problema

Criar um programa que permita:

- validar notas;
 - calcular a média de um aluno;
 - classificar o aluno;
 - mostrar o resultado;
 - organizar o código em módulos.
-

19.2 Versão simples num só ficheiro

```
def validar_nota(nota):
    if nota < 0 or nota > 20:
        raise ValueError("A nota deve estar entre 0 e 20")

def calcular_media(notas):
    if len(notas) == 0:
        raise ValueError("A lista de notas não pode estar vazia")

    for nota in notas:
        validar_nota(nota)

    return sum(notas) / len(notas)

def classificar(media):
    if media >= 18:
        return "Excelente"
    if media >= 14:
        return "Bom"
    if media >= 10:
        return "Suficiente"
    return "Insuficiente"

notas = [14, 16, 15]
media = calcular_media(notas)
classificacao = classificar(media)

print("Média:", media)
print("Classificação:", classificacao)
```

19.3 Estrutura modular

```
gestao_notas/
    main.py
    modelo.py
    vista.py
    controlador.py
```

19.4 modelo.py

```
def validar_nota(nota: float) -> None:
    """Valida se uma nota está entre 0 e 20."""
    if nota < 0 or nota > 20:
        raise ValueError("A nota deve estar entre 0 e 20")

def calcular_media(notas: list[float]) -> float:
    """Calcula a média de uma lista de notas válidas."""
    if len(notas) == 0:
        raise ValueError("A lista de notas não pode estar vazia")

    for nota in notas:
        validar_nota(nota)

    return sum(notas) / len(notas)

def classificar(media: float) -> str:
    """Classifica o aluno com base na média."""
    if media >= 18:
        return "Excelente"
    if media >= 14:
        return "Bom"
    if media >= 10:
        return "Suficiente"
    return "Insuficiente"
```

19.5 vista.py

```
def pedir_notas() -> list[float]:
    """Pede notas ao utilizador até ser introduzido texto vazio."""
    notas = []

    while True:
        texto = input("Introduz uma nota ou Enter para terminar: ")

        if texto == "":
            break

        try:
            nota = float(texto)
            notas.append(nota)
        except ValueError:
            print("Valor inválido. Introduz um número.")

    return notas

def mostrar_resultado(media: float, classificacao: str) -> None:
    """Mostra o resultado final."""
```



```
print(f"Média: {media:.2f}")
print(f"Classificação: {classificacao}")
```

```
def mostrar_erro(mensagem: str) -> None:
    """Mostra uma mensagem de erro."""
    print("Erro:", mensagem)
```

19.6 controlador.py

```
from modelo import calcular_media, classificar
from vista import pedir_notas, mostrar_resultado, mostrar_erro
```

```
def executar() -> None:
    """Coordena o funcionamento do programa."""
    notas = pedir_notas()

    try:
        media = calcular_media(notas)
        classificacao = classificar(media)
        mostrar_resultado(media, classificacao)
    except ValueError as erro:
        mostrar_erro(str(erro))
```

19.7 main.py

```
from controlador import executar
```

```
if __name__ == "__main__":
    executar()
```

19.8 Vantagens da versão modular

A versão modular permite:

- alterar a forma de mostrar dados sem mexer nos cálculos;
 - testar `calcular_media()` separadamente;
 - reutilizar `classificar()` noutros programas;
 - manter o programa organizado;
 - aumentar o projeto com novas funcionalidades.
-

19.9 Possíveis melhorias ao estudo de caso

O programa poderia ser melhorado para:

- guardar notas em ficheiro CSV;
- carregar dados de um ficheiro;
- calcular média por turma;

- mostrar a melhor e a pior nota;
 - criar gráficos com matplotlib;
 - criar uma interface gráfica;
 - criar testes automáticos.
-

20. Resumo geral

20.1 Ideias principais

- Funções organizam programas.
 - `def` cria funções.
 - Parâmetros recebem dados.
 - Argumentos são valores enviados à função.
 - `return` devolve resultados.
 - `print()` mostra informação no ecrã.
 - Validação evita erros.
 - Exceções permitem tratar situações problemáticas.
 - Objetos mutáveis podem ser alterados dentro de funções.
 - Funções pequenas são mais fáceis de testar.
 - Bibliotecas reutilizam código já existente.
 - Módulos são ficheiros Python.
 - Pacotes são conjuntos de módulos.
 - Modularização melhora a manutenção.
-

20.2 Tabela rápida

Conceito	Descrição curta
Função	Bloco reutilizável de código.
Parâmetro	Variável na definição da função.
Argumento	Valor enviado na chamada.
<code>return</code>	Devolve um valor.
<code>print()</code>	Mostra informação.
Exceção	Erro que pode ser tratado.
Módulo	Ficheiro Python reutilizável.
Biblioteca	Conjunto de código reutilizável.
Pacote	Pasta com módulos.
<code>import</code>	Importa módulos ou bibliotecas.

21. Exercícios propostos

21.1 Exercícios básicos

1. Cria uma função `saudacao()` que mostre a mensagem Olá, Python!.
2. Cria uma função `dobro(numero)` que devolva o dobro de um número.
3. Cria uma função `somar(a, b)` que devolva a soma de dois números.
4. Cria uma função `area_retangulo(largura, altura)`.
5. Cria uma função `converter_celsius_para_fahrenheit(celsius)`.
6. Cria uma função `maior(a, b)` que devolva o maior de dois números.

7. Cria uma função `eh_par(numero)` que devolva `True` se o número for par.
 8. Cria uma função `contar_caracteres(texto)`.
 9. Cria uma função `primeiro_elemento(lista)`.
 10. Cria uma função `ultimo_elemento(lista)`.
-

21.2 Exercícios intermédios

1. Cria uma função `calcular_media(notas)` que valide se a lista não está vazia.
 2. Cria uma função `validar_nota(nota)` que aceite apenas notas entre 0 e 20.
 3. Cria uma função `classificar(media)` que devolva `Aprovado` ou `Reprovado`.
 4. Cria uma função `contar_pares(numeros)`.
 5. Cria uma função `filtrar_positivos(numeros)`.
 6. Cria uma função `remover_repetidos(lista)`.
 7. Cria uma função `inverter_texto(texto)`.
 8. Cria uma função `eh_palindromo(texto)`.
 9. Cria uma função `fatorial(n)` com validação.
 10. Cria uma função `potencias(numeros, expoente)` usando list comprehension.
-

21.3 Exercícios avançados

1. Cria um programa modular para gerir notas de alunos.
 2. Cria um módulo `conversoes.py` com funções de conversão de temperatura e distância.
 3. Cria um módulo `estatistica.py` com média, máximo, mínimo e amplitude.
 4. Cria uma função que leia dados de um ficheiro CSV.
 5. Cria uma função que guarde resultados num ficheiro JSON.
 6. Usa `map()` para converter uma lista de temperaturas em Celsius para Fahrenheit.
 7. Usa `filter()` para selecionar apenas notas positivas.
 8. Usa `sorted()` com `lambda` para ordenar alunos por nota.
 9. Cria testes com `assert` para três funções.
 10. Melhora o estudo de caso com gravação em ficheiro.
-

21.4 Mini-projeto: calculadora modular

Cria um projeto com a seguinte estrutura:

```
calculadora/  
  main.py  
  operacoes.py  
  vista.py
```

Requisitos:

- `operacoes.py` deve ter funções para somar, subtrair, multiplicar e dividir;
 - a divisão deve tratar divisão por zero;
 - `vista.py` deve pedir dados ao utilizador e mostrar resultados;
 - `main.py` deve iniciar o programa.
-

21.5 Mini-projeto: análise de turma

Cria um programa que receba uma lista de alunos com notas.

O programa deve calcular:

- média da turma;
 - melhor nota;
 - pior nota;
 - número de alunos aprovados;
 - número de alunos reprovados;
 - lista ordenada por nota.
-

22. Soluções orientadoras

22.1 Soluções básicas

Exercício: dobro

```
def dobro(numero):  
    return numero * 2
```

Exercício: área do retângulo

```
def area_retangulo(largura, altura):  
    return largura * altura
```

Exercício: verificar par

```
def eh_par(numero):  
    return numero % 2 == 0
```

22.2 Soluções intermédias

Média com validação

```
def calcular_media(notas):  
    if len(notas) == 0:  
        raise ValueError("A lista de notas não pode estar vazia")  
    return sum(notas) / len(notas)
```

Validar nota

```
def validar_nota(nota):  
    if nota < 0 or nota > 20:  
        raise ValueError("A nota deve estar entre 0 e 20")
```

Classificar média

```
def classificar(media):  
    if media >= 10:  
        return "Aprovado"  
    return "Reprovado"
```

Filtrar positivos

```
def filtrar_positivos(numeros):  
    return [numero for numero in numeros if numero > 0]
```

Remover repetidos mantendo ordem

```
def remover_repetidos(lista):  
    resultado = []  
  
    for elemento in lista:  
        if elemento not in resultado:  
            resultado.append(elemento)  
  
    return resultado
```

Palíndromo

```
def eh_palindromo(texto):  
    texto = texto.lower().replace(" ", "")  
    return texto == texto[::-1]
```

22.3 Solução avançada: ordenar alunos por nota

```
alunos = [  
    {"nome": "Ana", "nota": 15},  
    {"nome": "Bruno", "nota": 12},  
    {"nome": "Carla", "nota": 18},  
]  
  
ordenados = sorted(alunos, key=lambda aluno: aluno["nota"], reverse=True)  
print(ordenados)
```

22.4 Solução avançada: guardar JSON

```
import json  
from pathlib import Path  
  
def guardar_json(dados, caminho):  
    texto = json.dumps(dados, ensure_ascii=False, indent=4)  
    Path(caminho).write_text(texto, encoding="utf-8")  
  
alunos = [  
    {"nome": "Ana", "nota": 15},  
    {"nome": "Bruno", "nota": 12},  
]  
  
guardar_json(alunos, "alunos.json")
```

23. Anexos

23.1 Tabela de comandos essenciais

Comando	Descrição
def	Cria uma função.
return	Devolve um valor.
print()	Mostra informação.
raise	Lança uma exceção.
try	Inicia bloco de tentativa.
except	Captura erro.
else	Executa se não houver erro.
finally	Executa sempre.
import	Importa módulo ou biblioteca.
from ... import ...	Importa parte específica de um módulo.
lambda	Cria função anónima curta.
assert	Verifica uma condição em testes simples.

23.2 Glossário

Termo	Significado
Função	Bloco reutilizável de código.
Procedimento	Função usada principalmente para executar ações.
Parâmetro	Variável usada na definição da função.
Argumento	Valor enviado à função.
return	Palavra-chave que devolve um resultado.
None	Valor especial que representa ausência de valor.
Exceção	Erro que pode ser tratado pelo programa.
Mutável	Objeto que pode ser alterado depois de criado.
Imutável	Objeto que não pode ser alterado depois de criado.
Biblioteca	Conjunto reutilizável de código.
Módulo	Ficheiro Python reutilizável.
Pacote	Pasta com módulos.
Modularização	Divisão de um programa em partes organizadas.
Docstring	Texto que documenta uma função.
Type hint	Indicação do tipo esperado numa função.

23.3 Erros comuns e correções

Erro	Exemplo	Correção
Esquecer parênteses	funcao	funcao()
Falta de indentação	código desalinhado	usar 4 espaços
Falta de return	função devolve None	adicionar return
Lista vazia	sum([]) / len([])	validar antes
Divisão por zero	a / 0	verificar divisor
Import errado	import matematica sem ficheiro	confirmar nome e localização

Erro	Exemplo	Correção
Alterar lista sem querer	<code>lista.append()</code> dentro da função	usar cópia da lista

23.4 Plano de estudo sugerido

1. Criar funções simples sem parâmetros.
2. Criar funções com parâmetros.
3. Treinar return e guardar resultados.
4. Comparar print() e return.
5. Validar dados com if.
6. Tratar erros com try e except.
7. Usar listas e dicionários em funções.
8. Praticar mutabilidade.
9. Usar bibliotecas padrão como math, random e datetime.
10. Criar módulos próprios.
11. Modularizar um pequeno projeto.
12. Escrever testes simples com assert.

23.5 Checklist final antes de entregar um programa

- O programa corre sem erros?
- As funções têm nomes claros?
- Há funções demasiado grandes?
- Os dados principais são validados?
- As exceções importantes são tratadas?
- O código repetido foi transformado em funções?
- Os cálculos estão separados da apresentação?
- As bibliotecas foram importadas corretamente?
- Há pelo menos alguns testes simples?
- O programa está organizado em módulos quando necessário?

Notas finais

Esta sebenta foi expandida para servir como material de estudo e prática sobre funções e bibliotecas em Python. Para aprender bem este tema, é essencial escrever código, testar exemplos, cometer erros e corrigi-los.

A programação melhora com prática constante e com a resolução de problemas reais.